

Autonomous Sueca Card Game AI

A Machine Learning Approach

Diogo Nascimento nº120031

Gonçalo Calvo nº120131

Tiago Mendes nº119773

Abstract

This presentation will detail the development of a machine learning model designed to autonomously play the card game “Sueca”.

We outline the data processing pipeline implemented to transform raw match records into turn-by-turn vectors, present our stratified validation strategy, and evaluate our chosen models which include Logistic Regression, Gradient Boosting, and XGBoost.

Project Overview

An artificial intelligence agent for the card game “Sueca” that learns from a dataset with information from generated matches of other agents that play cards randomly (following the rules of the game). Our goal was to tackle this as a classification problem, where our model must choose the correct card out of the ones in its hand to play.

Dataset Structure (Before)

Trick → Row

Messy Data Structure

**Lack of individual
perspective**

- Each row having a full tricks gives first plays a disadvantage
- Models have a hard time reading data that is not well formatted (like strings or JSON structures)
- Since we aren't tracking individual turns, it's harder for a model to learn

Dataset Structure (After)

Trick → Row

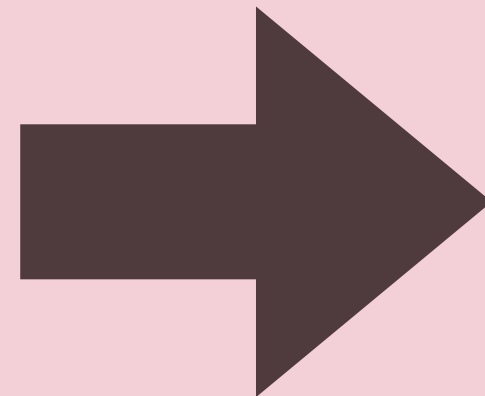
Turn → Row

Messy Data Structure

Unscramble Data

Lack of individual
perspective

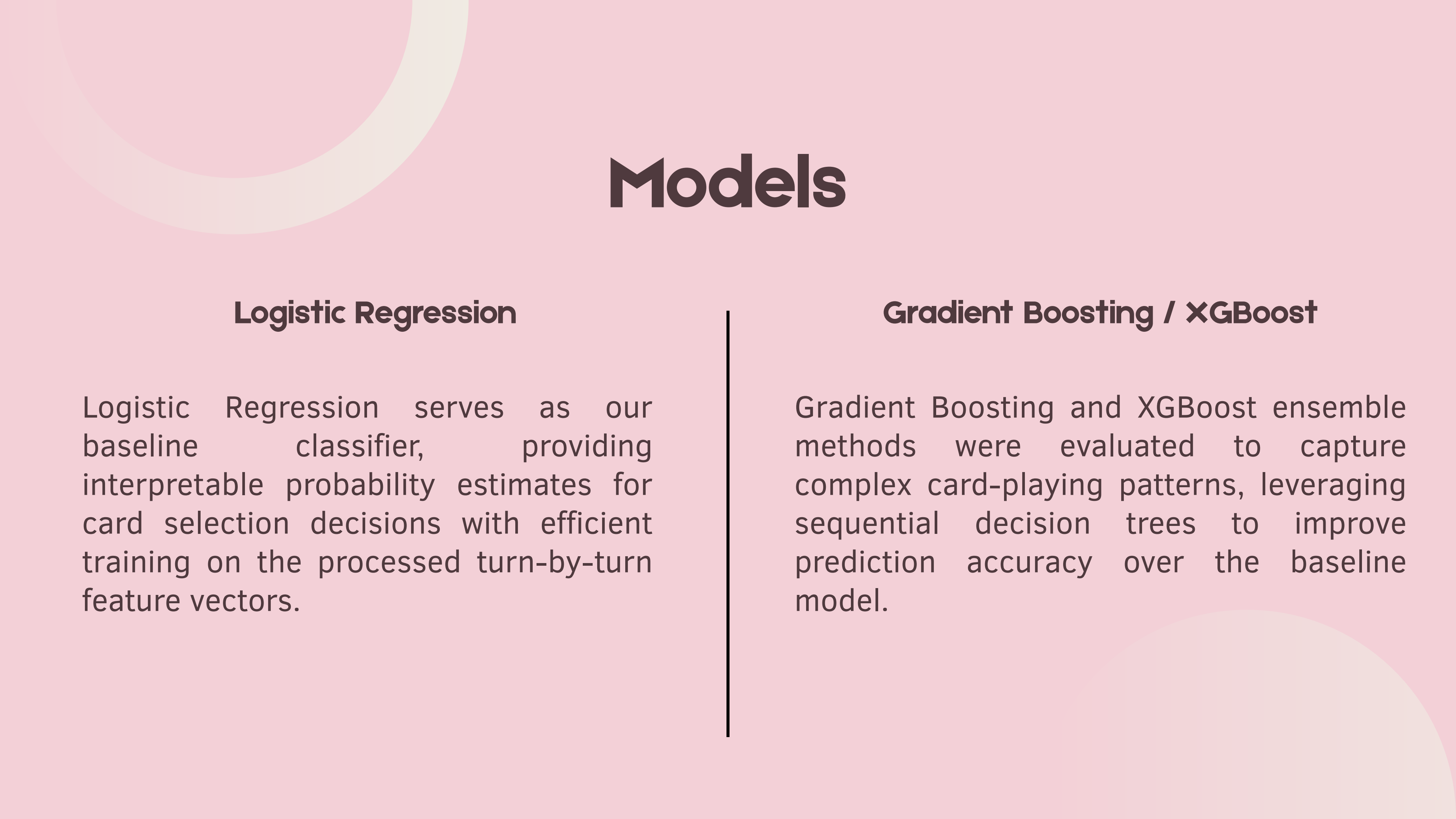
Clear individual
goals (Target)



Validation Strategy

Stratified K-Fold Cross Validation

We employed stratified sampling to ensure each fold maintains the same class distribution as the full dataset, preventing bias in model evaluation across different game outcomes.



Models

Logistic Regression

Logistic Regression serves as our baseline classifier, providing interpretable probability estimates for card selection decisions with efficient training on the processed turn-by-turn feature vectors.

Gradient Boosting / XGBoost

Gradient Boosting and XGBoost ensemble methods were evaluated to capture complex card-playing patterns, leveraging sequential decision trees to improve prediction accuracy over the baseline model.

Models (Metrics)

Accuracy

Log Loss

Top-3 Accuracy

Models (Methodology)

Compare the 3 Models

We created a file (model_comparison.py) that allows us to compare the 3 models.

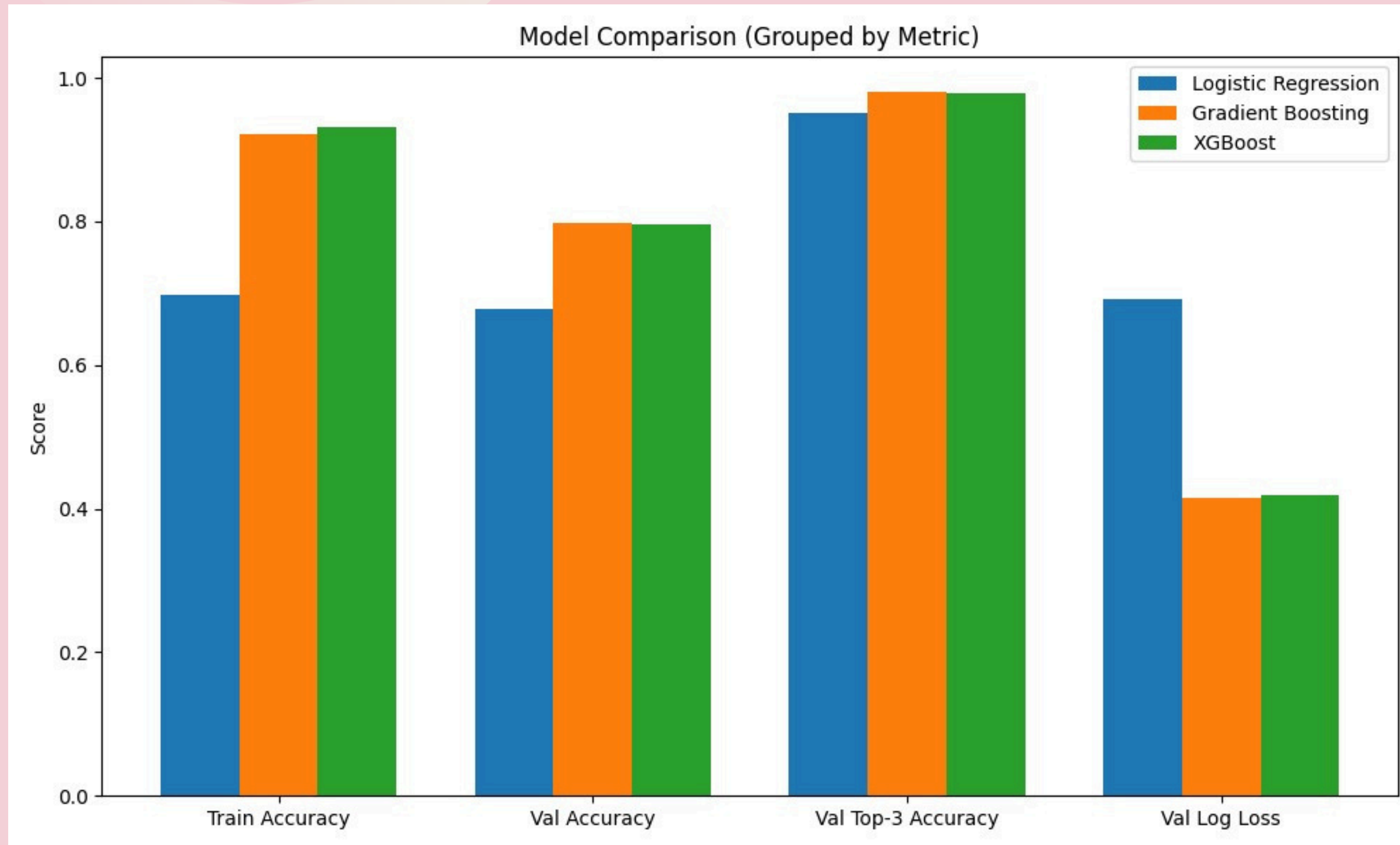
Individual Grid Search

Each model gets their own grid search file, in order to determine the best hyperparameters to test them with

Compare more

After grid search, we apply the new parameters to the models and rerun them together.

Models (Results)



Logistic Regression
Accuracy: 0.6785

Gradient Boosting
Accuracy: 0.7971

XGBoost
Accuracy: 0.7984

Models (Tuning)

Logistic Regression

- `logisticregression__C`
- `logisticregression__solver`

- 18 Combinations
- 90 CV runs

GridSearchCV

Gradient Boosting

- `learning_rate`
- `max_depth/max_leaf_nodes`
- `min_samples_leaf`

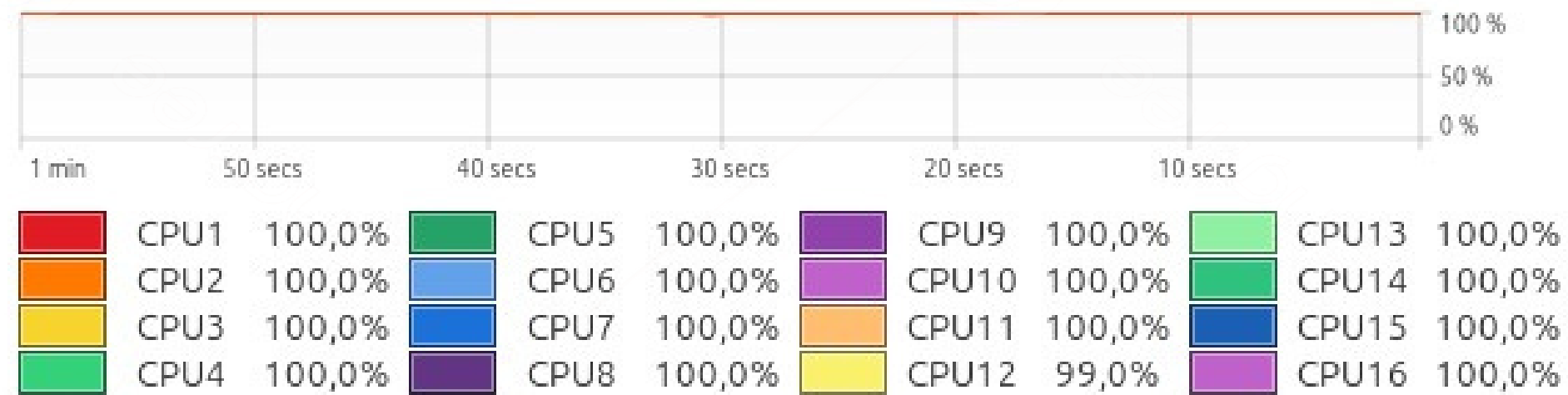
- 24 Combinations
- 120 CV runs

XGBoost

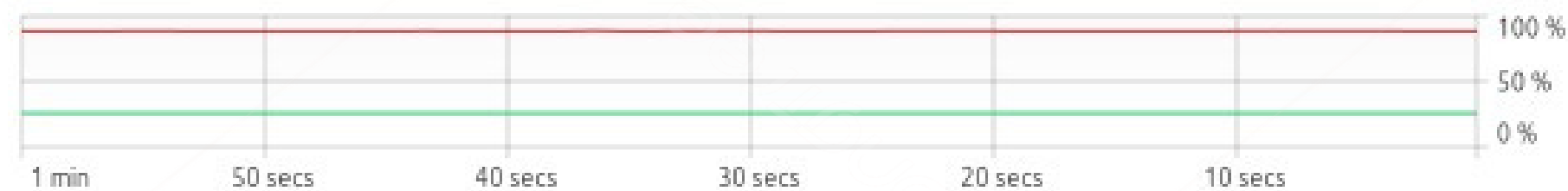
- `n_estimators`
- `max_depth`

- 4 Combinations
- 20 CV runs

▼ CPU



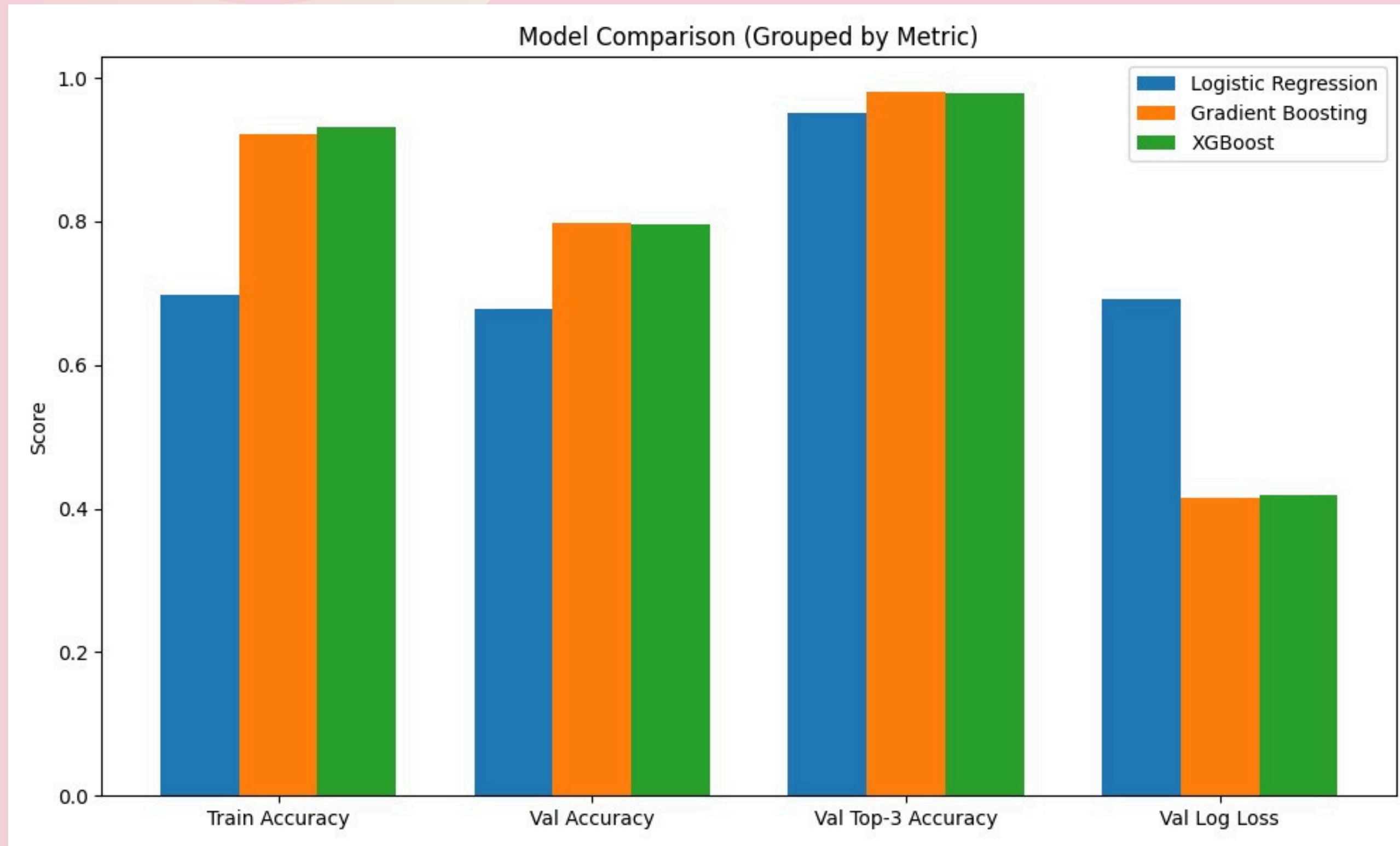
▼ Memory and Swap



 Memory
29,1 GB (88,5%) of 32,8 GB
Cache 4,5 GB

 Swap
4,5 GB (25,9%) of 17,2 GB

Models (Results)

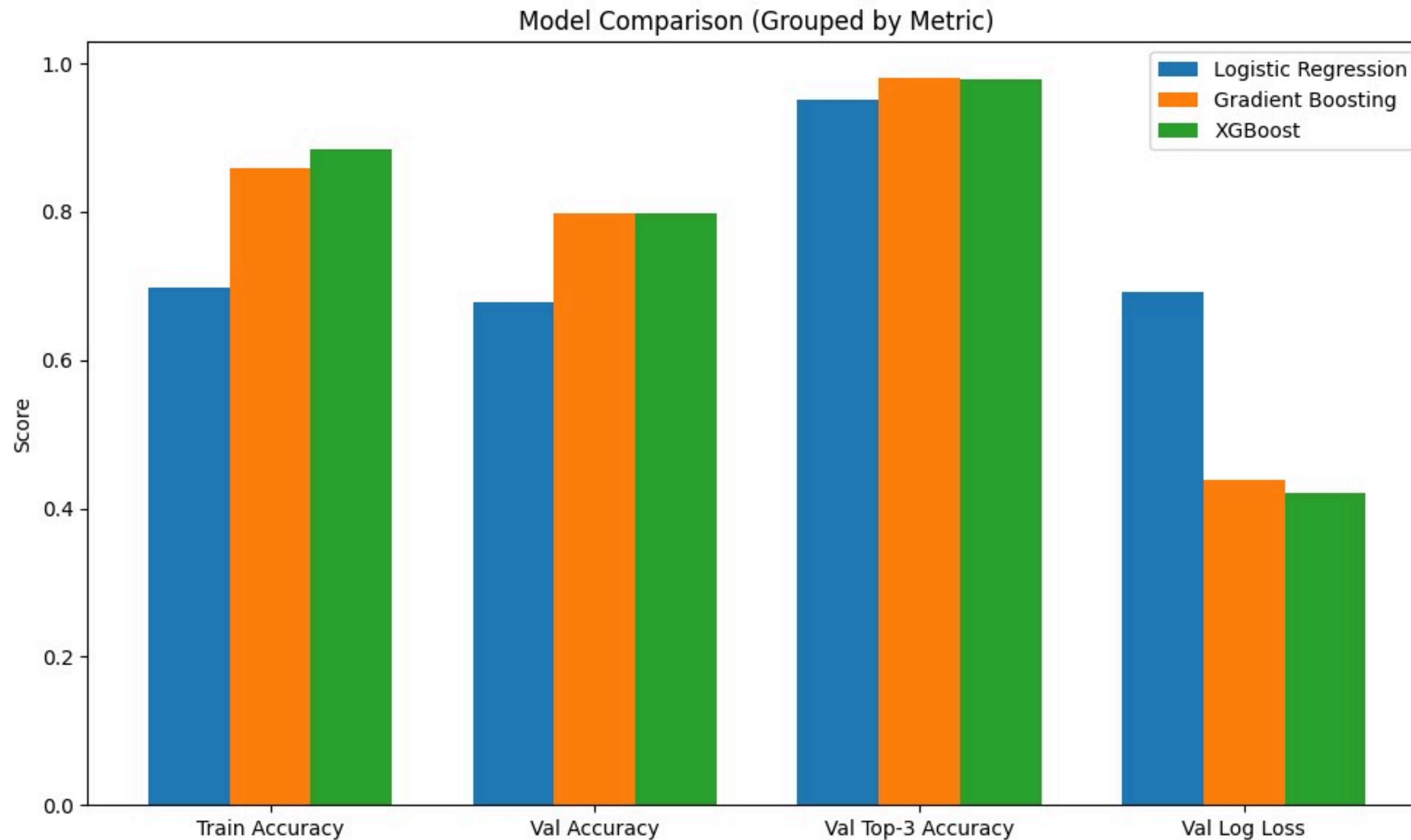


Logistic Regression
Accuracy: 0.6785

Gradient Boosting
Accuracy: 0.7971

XGBoost
Accuracy: 0.7984

Models (Results)



Logistic Regression
Accuracy: 0.6788

Gradient Boosting
Accuracy: 0.7995

XGBoost
Accuracy: 0.7973

Models (Interpretation)

- **Logistic Regression < Gradient Boosting =? XGBoost**
- **Across both iterations, Logistic Regression seems to be consistently underfitting**
- **At the beginning, Gradient Boosting and XGBoost (our best candidate) were overfitting**
- **Even though XGBoost lost some Train Accuracy, it maintained its Validation Accuracy, indicating positive growth (reduced overfitting)**

Models (Improvements)

- Using k-3 accuracy as the main metric for grid search was unhelpful. Switching to accuracy may be better for overall results
- Our choice of parameters was not as well thoughtout, as we jump into more detailed hyper parameter tuning for XGBoost, we would try to study our options better, specifically to try to better solve the overfitting problem

Ethical Considerations

- Our models were not learning the best moves to play, but instead mimicking the plays of whatever generated our dataset
- If we want to later use better data, we would probably require data from actual player, playing actual strategies, which could cause problems with the privacy and ownership of the data
- Assuming we can keep scaling our model, we may run into the obvious problem of it being used for cheating and reducing the competitive integrity of games

Conclusions

- **XGBoost and Gradient Boosting ensemble models demonstrated superior classification performance compared to baseline Logistic Regression, achieving more accurate predictions for optimal card play in Sueca game states.**
- **The stratified validation strategy confirmed model generalization across diverse match scenarios, with the data pipeline successfully transforming raw match records into meaningful turn-by-turn feature vectors for training.**
- **We would require more time to do a more thorough investigation hyperparameters, preferably solely on XGBoost, as it is our best performing option, and less time spent on other models is more time spent on this one**

Thank You